Checks That Can Fail

Assured AI agents and surrogate models for faster ground-system design and simulation

[Your name], M.S. Cybersecurity · Research plan for the [Org] AI Research Fellow, UM-LANL SPARC · October 2026

Summary. AI agents and surrogate models can speed vehicle design, but only as far as the checks that accept their work can be trusted, and checks fail. In a two-week pilot on my robot, Scorpion, every check was itself tested: planted faults for CAD checks, the simulator for the surrogate, and real drops for the simulator. A check has been caught being wrong 19 times, never by the check itself. This plan makes it routine.

1. THE CHALLENGE

The slow step in building a vehicle is not drawing a change but building and testing it. AI agents can now revise CAD, set up simulations and write the tests that judge the result in minutes, yet each change still waits on hardware: on my robot, 16 of 26 design flaws surfaced only at the bench, after printing. In this pilot, 361 hand drops took three sessions; the simulator ran 80,000 in 74 s, and a surrogate model answers in microseconds. Simulation and surrogates can replace most builds, but only if their answers can be trusted. A check that passes when it should fail sends a defect to hardware, where each one costs a reprint and a bench session; a check nobody trusts sends every design back to the bench, and the speed is lost.

I propose to make that acceleration *assured*: to give AI-agent design changes, and the physics and surrogate checks that accept them, measured confidence, while keeping a same-day design, build and test loop. This is the priority because it gates every other use of AI in vehicle engineering, it can be measured, and it transfers across subsystems. It is also an assurance problem of the kind cybersecurity trains for: assume any control can be fooled, by a bug or by the system's own mistake, and test the tests.

2. PRELIMINARY WORK

I have been building **Scorpion**, a tracked ground robot with a 360° pan-tilt camera head, as a platform for embodied AI since February 2025. Since July 2026 I have worked on it with an AI coding agent (Claude, by Anthropic): first its software, and since September its CAD. I build and test at the bench; the agent turns each finding into a change to code-based CAD in Python (CadQuery and signed-distance models), runs automated checks and produces print files, often the same day.

- **Pace.** The pan drive went through 11 design iterations in 16 days, the last four on consecutive days.
- **Checks.** Interference, assembly order, gear backlash, bearing contact, thread engagement and wire clearance; for the whole vehicle, stability, rest states after a tumble, and which part strikes first.
- **Finding.** Ten recorded cases of a check that was itself wrong, seven of them passes that let a defect through. The fix that worked: every check ships with a *negative control*, a deliberately broken variant it must flag, which is mutation testing borrowed from software engineering [1].
- **Escapes.** A lesson alone did not stop a fault's kind: a screwdriver-access rule (Sep 18) let an access fault reach the bench (Oct 3). A lesson with a check did: after Oct 3 the one assembly fault designed (Oct 11) was caught by its check before printing. What kept escaping is what no check models yet: forces (8 of 16 bench findings: friction, fit, strength, retention, material) and wire routing (5 findings).

3. PILOT RESULTS

To turn those anecdotes into measurements, the agent and I (below, "we") ran a two-week pilot of each aim on Scorpion in October 2026. H2's and H3's predictions were committed to version control before their tests [7], and H1's 22 faults before they were run; H4 is retrospective. Each experiment has a control, a broken case that must come out differently, and every number here traces to a file in the repository.

- **H1. Mutation-tested checks (Aim 1).** We seeded 22 faults into the pan drive's model, 14 re-creating real failures (Fig. 1A). The first check, part-against-part interference, caught 12 (7 of the 14 real ones). The suite grown since, one check for each escape, caught 21. The one survivor, a part 0.6 mm out of its grooves, became a new check: 22 of 22. None of the ten checks is vacuous: each catches a fault.

- **H2. An uncertainty-gated surrogate (Aim 2).** A MuJoCo simulation [3] tumbled 400 body shapes 200 times each, 80,000 tumbles in 74 s on 20 cores, to find which shapes come to rest upright. A Gaussian process [5] trained on 120 of them predicted the other 280 with an RMS error of 3.9 percentage points, and its 90% intervals held 86.8% of the time (Fig. 1B). Asked whether each design lands upright at least 60% of the time, it answered 266 of 280 by itself, 2 of them wrongly, and sent 14 to simulation: 95% of the simulations saved. Two checks exposed its limits, and both are findings. A registered control failed: sentinel runs could not see a hidden friction change, which moved the answer by at most 4 points. And a later check found that intervals that held on average held for only 1 of the 6 best designs, all about 5 points too low: exactly where a design gets chosen.
- **H3. Simulation against reality.** We printed scale models of three candidate bodies, each with a coin pocket so its centre of mass can be moved as a physical control, and I dropped them by hand 361 times, from 6 and 30 cm. Each prediction is scored by its Brier score [6]: 0 is perfect, and lower is better (Fig. 1C). At 6 cm the committed predictions beat a naive guess, 0.58 against 1.29, and shifted the right way when the coins moved. That pass was partly luck. Breaking the misses down by starting position exposed a bug in how simulated drops began, and fixing it alone made the score worse (0.86). Watching drops in the simulator's viewer, I saw its default floor let bodies sink 7–17 mm on impact, which a passing audit had measured only at rest. A floor bounce fitted at 6 cm and committed before the 30 cm drops then won that blind test: 0.63 against 1.16 for the default floor (difference 0.52, 95% CI 0.40–0.64). After the fact, a hard floor with its bounce measured on 240 fps video comes close without fitting anything: 0.46 and 0.67, against a best possible near 0.44 and 0.60. A third blind test, at 15 cm from a drop stand, held: the registered primary, a hard floor with bounce 0.31, scored 0.68 against 1.12 for the default floor (difference 0.44, 95% CI 0.35–0.54); with the ball's measured bounce (0.39, Fig. 1C) it scored 0.64, near the best possible 0.61. At 15 and 30 cm the model still over-predicts the weight's effect.
- **H4. Cycle time.** Reconstructed from version control and logs (retrospective), the median from a finding to a fix ready to print was 1.6 h (interquartile range 0.6–2.5 h, $n = 20$; 1.9 h for bench findings); 10 of 26 findings were caught before printing, and 5 had slipped past a check that passed. During the pilot, 17 more findings were logged as they happened, with commit times.

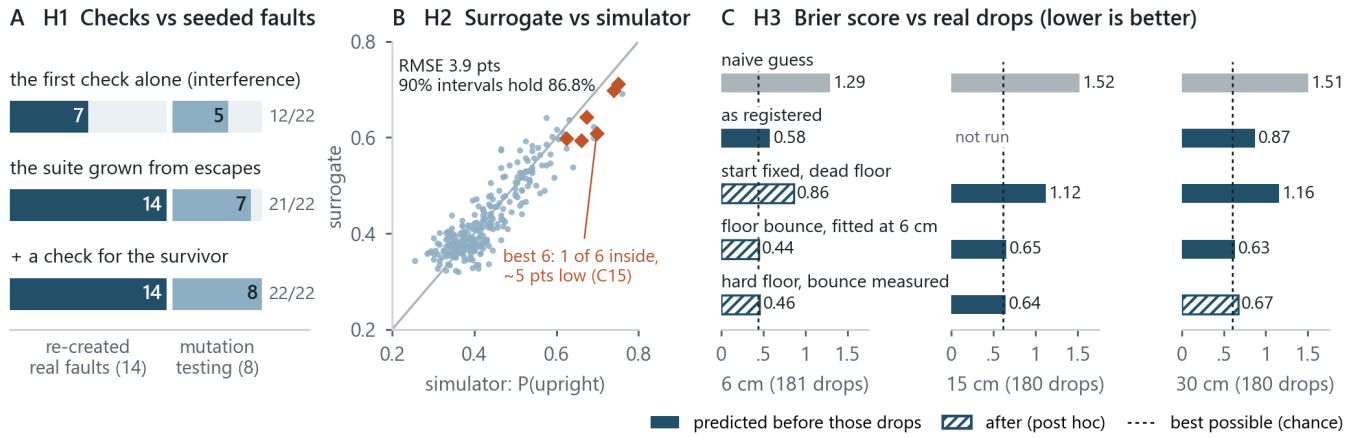


Figure 1. Pilot results. (A) Seeded faults caught: the 14 re-created real failures, and the 8 invented for mutation testing, the out-of-sample test. (B) Surrogate against simulator on 280 held-out designs; orange, the 6 best designs, each predicted without its own data, against 1,000-drop reruns. (C) Brier scores against the real drops: solid bars committed before those drops, hatched after (the 6 cm floor bounce was fitted to them); dashed, the best score the drops' own randomness allows.

What the pilot shows. The project has now recorded 19 cases of a check that was itself wrong, 13 of them passes that should have failed, and 9 of them during the pilot: in a CAD check suite and its controls, the drop predictions, the simulator's floor, a numerical time step and the surrogate's intervals. None was caught by the check itself. Each was exposed by something outside it: a person looking, a physical test, a cross-check, a control or a second agent, and each changed how that kind of check is done. That is the case for this plan: checks that can fail, and are shown to.

4. APPROACH

Aim 1: Mutation-tested agent workflows. Grow the pilot's harness into a fault-operator library (move a part, close a clearance, swap a material, sever a joint, duplicate a body, perturb a load) that scores each check by the faults it catches. The agent delivers each change with its checks; a second agent, blind to the designer's reasoning, writes the mutants; and a change passes only when they reach a set mutation score. Designs, results and logs are versioned and signed: a chain of custody for AI-generated designs.

Aim 2: Physics, surrogates and uncertainty. Couple the checks to physics-based simulation: Project Chrono for vehicle-terrain mobility and rollover [2], MuJoCo for mechanisms [3], assembly planning for whether a design goes together [8], and [Org] and SPARC tools where they fit. Train surrogates on simulation sweeps run on high-performance computing [4], so the agent can screen a change in seconds; send any case a surrogate is unsure of to full simulation or physical test; and check its intervals where decisions are made, not only on average. A surrogate is a check too, so it faces the same mutants. Bench data calibrates the physics, and each escape becomes a regression case and training data.

Aim 3: Demonstrate and measure. With the Chief Scientist, choose a ground-system problem from [Org] and SPARC priorities, for example a sensor mast, a drive module or rollover screening. Run a controlled comparison on matched tasks: the conventional workflow, AI agents alone, and AI agents with mutation-tested, uncertainty-gated checks. Measures: time from a test finding to a verified change; faults designed per change and the share caught before hardware, which separate prevention from detection; mutation score; whether the surrogate's stated uncertainty holds; compute and engineer hours.

5. SCHEDULE (12 MONTHS)

Months 1-2	Evaluate SPARC foundation models and codes; pick the problem; package the toolkit.
Months 3-6	Physics coupling, surrogate training on HPC; uncertainty gating; calibration; provenance logging.
Months 7-10	Controlled study on the selected subsystem.
Months 11-12	Analysis, transition package, technical report and publication.

6. DELIVERABLES, TRANSITION AND PARTNERSHIP

An open check-and-mutation library, in Python on open formats (STEP, STL, MJCF), that drops into existing CAD and simulation pipelines; trained surrogates with their validation records; a workflow guide and a half-day course for [Org] engineers; a dataset of design iterations, escapes and cycle times; a technical report and a paper. I will pair [Org] staff with UM and LANL experts in scientific machine learning and verification, and draft at least one joint follow-on proposal. Transition succeeds when a second team runs the harness on its own vehicle without me.

7. RISKS

- **Simulation and surrogate fidelity.** The pilot found both wrong in ways only outside checks exposed. Physical test stays the arbiter, and uncertainty gating limits where a surrogate may decide alone.
- **Agent errors.** No change is accepted without passing its mutants and a human review gate.
- **Check rot.** Libraries gather checks that stop catching faults or cry wolf (the pilot found a dead control); each keeps its mutation score and bench record, and is retired on that evidence, never silently.
- **Scale.** Shown on a small robot so far; Aim 3 tests it on an [Org] subsystem, with its own tools.

REFERENCES

[1] Y. Jia and M. Harman, "An analysis and survey of the development of mutation testing," *IEEE Trans. Softw. Eng.*, 37(5), 2011.
[2] A. Tasora et al., "Chrono: An open source multi-physics dynamics engine," *HPCSE 2015*, LNCS 9611, Springer, 2016.
[3] E. Todorov, T. Erez and Y. Tassa, "MuJoCo: A physics engine for model-based control," *IEEE/RSJ IROS*, 2012.
[4] G. E. Karniadakis et al., "Physics-informed machine learning," *Nature Reviews Physics*, 3, 2021.
[5] C. E. Rasmussen and C. K. I. Williams, *Gaussian Processes for Machine Learning*, MIT Press, 2006.
[6] G. W. Brier, "Verification of forecasts expressed in terms of probability," *Monthly Weather Review*, 78(1), 1950.
[7] B. A. Nosek, C. R. Ebersole, A. C. DeHaven and D. T. Mellor, "The preregistration revolution," *PNAS*, 115(11), 2018.
[8] Y. Tian et al., "Assemble Them All: Physics-based planning for generalizable assembly by disassembly," *ACM TOG* 41(6), 2022.